

Artificial Intelligence (AI-2002)

Lecture 9: Local Search and Optimization Problems

Mr. Saif ur Rehman (Lecturer – AI & DS)

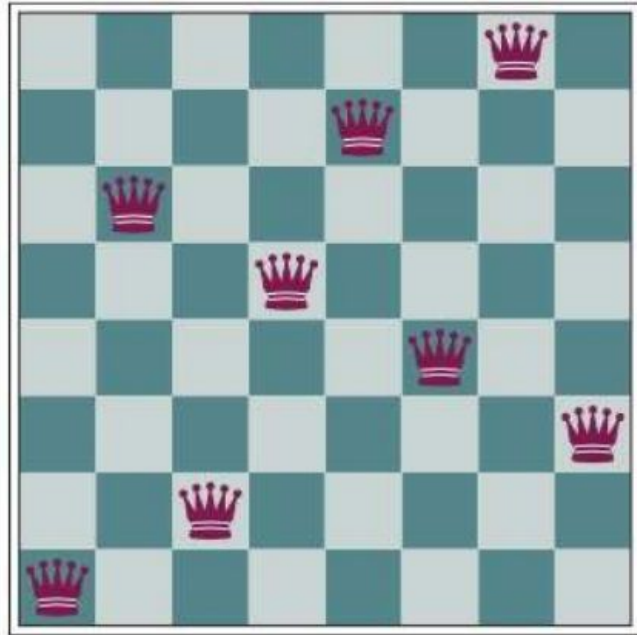
saifurrehman@nu.edu.pk



Local Search and Optimization Problems

- In previous search problems we wanted to find paths through the search space.
- Sometimes we care only about the final state, not the path to get there.
- For example, in the 8-queens problem, we care only about finding a valid final configuration of 8 queens

Local Search and Optimization Problems



(a)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	13	16	13	16	16
17	14	17	15	14	16	16	16
18	14	15	15	14	16	16	16
14	14	13	17	12	14	12	18

(b)

Local Search and Optimization Problems

- Local search algorithms operate by searching from a start state to neighboring states.
- Without keeping track of the paths, nor the set of states that have been reached.
- They might never explore a portion of the search space where a solution actually resides.
- However, they have two key advantages:
 - a. They use very little memory
 - b. They can often find reasonable solutions in large or infinite state spaces for which systematic algorithms are unsuitable.

Local Search and Optimization Problems

- Local search algorithms can also solve optimization problems, in which the aim is to find the best state according to an objective function.
- If aim is to find the highest peak—a global maximum—and we call the process hill Global maximum climbing.
- If aim is to find the lowest valley—a global minimum—and we call it gradient descent.

Local Search and Optimization Problems

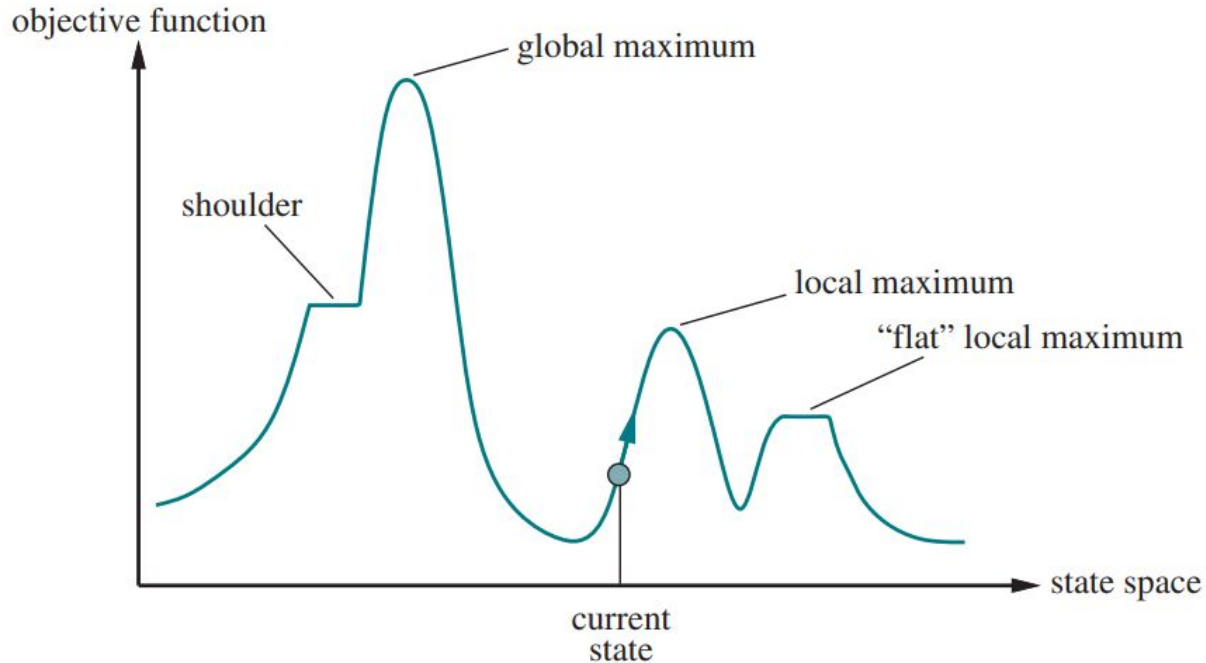


Figure 4.1 A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum.

Hill-climbing search

```
function HILL-CLIMBING(problem) returns a state that is a local maximum  
  current  $\leftarrow$  problem.INITIAL  
  while true do  
    neighbor  $\leftarrow$  a highest-valued successor state of current  
    if VALUE(neighbor)  $\leq$  VALUE(current) then return current  
    current  $\leftarrow$  neighbor
```

Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor.

Hill-climbing search

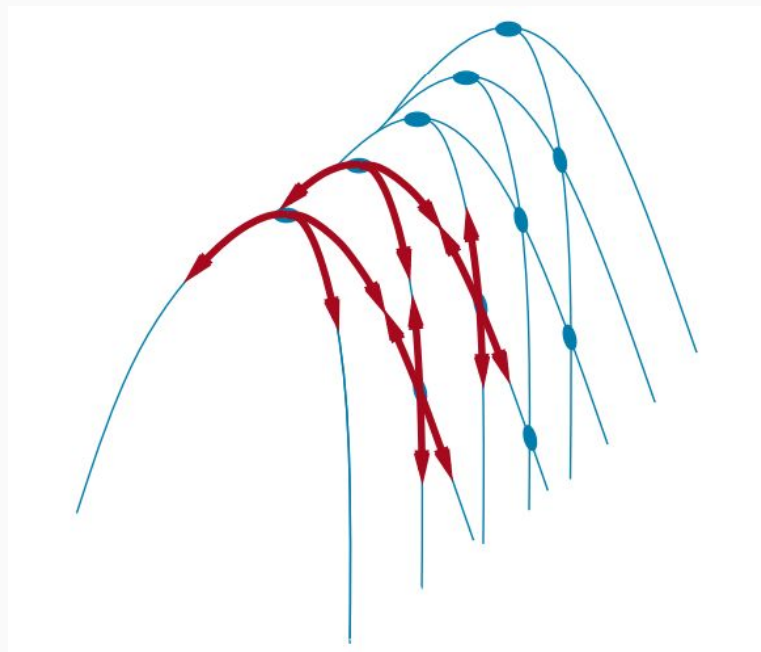
- Consider the 8-queens problem.
- The initial state is chosen at random.
- the successors of a state are all possible states generated by moving a single queen to another square in the same column (so each state has $8 \times 7 = 56$ successors).
- The heuristic cost function h is the number of pairs of queens that are attacking each other.
- this will be zero only for solutions.

Hill-climbing search

- Hill climbing is sometimes called greedy local search.
- Hill climbing can get stuck for any of the following reasons.
 - a. **Local maxima:** A local maximum is a peak that is higher than each of its neighboring states but lower than the global maximum.

Hill-climbing search

- **Ridges:** A ridge is shown in Figure. Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.
- **Plateaus:** A plateau is a flat area of the state-space landscape. It can be a flat local maximum, from which no uphill exit exists, or a shoulder, from which progress is possible.



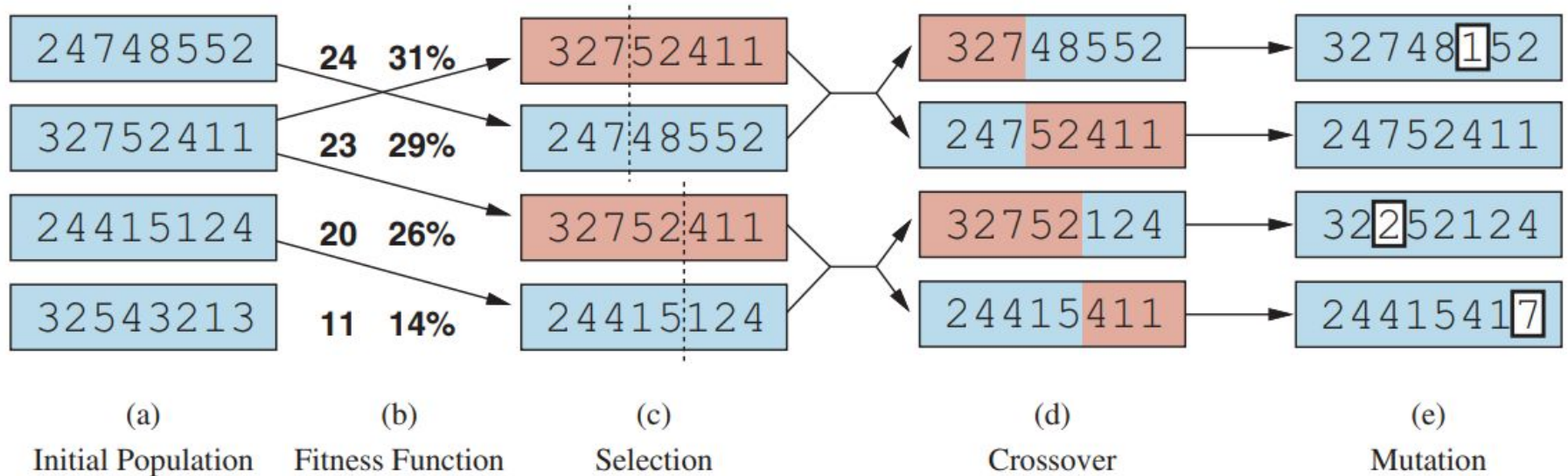
Hill-climbing search

- In each case, the algorithm reaches a point at which no progress is being made.
- Starting from a randomly generated 8-queens state, gets stuck 86% of the time, solving only 14% of problem instances.
- On the other hand, it works quickly, taking just 4 steps on average when it succeeds and 3 when it gets stuck.
- If consider the shoulder, s the percentage of problem instances solved by hill climbing from 14% to 94%.
- Success comes at a cost: the algorithm averages roughly 21 steps for each successful instance and 64 for each failure.

Genetic Algorithm

- Inspired by the principles of natural selection and evolution.

Genetic Algorithm



Genetic Algorithm

$$\text{Expected Count} = \frac{f(x_i)}{\text{Avg}(\sum f(x))}$$

String No.	Initial Population (Randomly Selected)	X Value	Fitness $f(x) = x^2$	Prob	% Prob	Expected Count	Actual Count
1	01100	12					
2	11001	25					
3	00101	5					
4	10011	19					
Sum							
Average							
Maximum							

Genetic Algorithm

String No.	Mating Pool	Crossover Point	Offspring after crossover	X Value	Fitness $f(x) = x^2$
1	01100				
2	11001				
3	11001				
4	10011				
Sum					
Average					
Maximum					